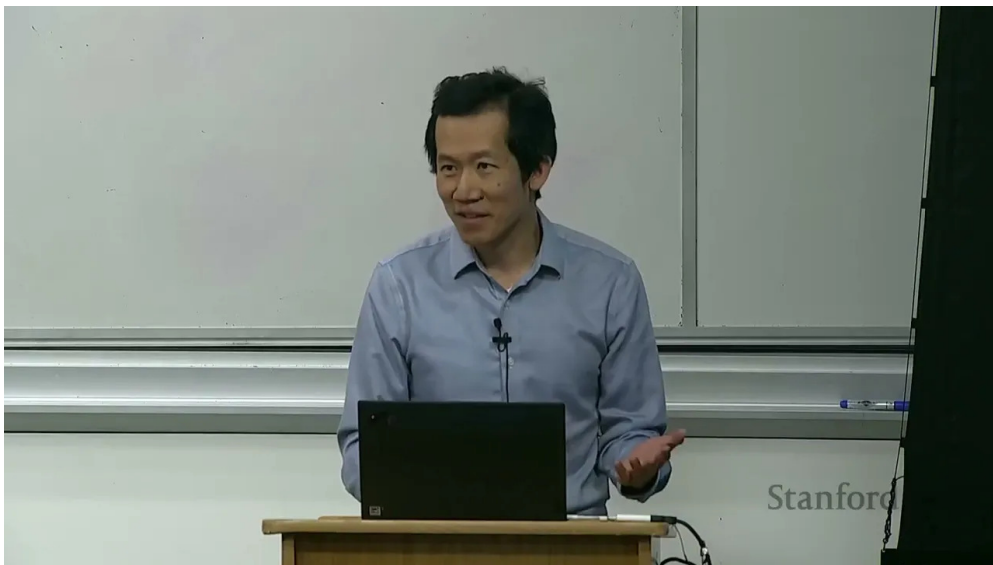


课程笔记

CS336 Lecture 12: Evaluation

基于 Tatsu Hashimoto 授课内容整理

2025 年 5 月



视频作者/频道: Stanford Online

发布日期: 2025-05

视频时长: 1:20:45

视频链接: <https://www.youtube.com/watch?v=>

目录

1 引言：为什么评估会变成一门大课	2
1.1 你平时看到的评估长什么样	2
2 如何理解评估	2
2.1 目标先于指标	3
2.2 输入、调用、输出、解释	3
3 Perplexity	3
3.1 为什么 perplexity 仍然重要	3
3.2 几种“精神上接近 perplexity”的任务	4
4 Knowledge Benchmarks	4
4.1 MMLU	4
4.2 MMLU-Pro	4
4.3 GPQA 和 Humanity’s Last Exam	4
5 Instruction Following	5
5.1 Chatbot Arena	5
5.2 IFEval	5
5.3 AlpacaEval 和 WildBench	5
6 Agent Benchmarks	5
6.1 SWE-bench	6
6.2 CyBench	6
6.3 MLEBench	6
7 Pure Reasoning	6
7.1 ARC-AGI	6
8 Safety Benchmarks	6
8.1 HarmBench 与 AIR-Bench	6
8.2 Jailbreaking	7
8.3 Pre-deployment testing	7
8.4 什么是安全	7
9 Realism	7
9.1 Quizzing vs Asking	7
9.2 Clio 和 MedHELM	7
10 Validity	8
10.1 Train-test overlap	8
10.2 Dataset quality	8

11 我们到底在评估什么	8
11.1 方法还是系统	8
11.2 为什么这很重要	8
11.3 几个例外	8
12 总结	9

1 引言：为什么评估会变成一门大课

这节课讨论 **evaluation**。乍看之下，评估似乎很简单：给定一个固定模型，问它“有多好”，然后算一个分数就行了。实际情况远没有这么直接。评估不仅决定了论文表格里的数字，也在很大程度上决定了模型开发、产品选择、政策判断和安全测试的方向。

本课的中心观点

评估不是一个机械步骤，而是一个需要先定义目标、再定义输入、调用方式、输出度量与结果解释的完整系统设计问题。

课程一开始展示了几个很常见但含义并不完全相同的榜单：模型在 MMLU、MATH、GPQA 等基准上的分数，HELM 的能力榜，Artificial Analysis 的能力-价格前沿，OpenRouter 的流量排行，以及 Chatbot Arena 的人类偏好排名。它们都在回答“模型好不好”，但回答角度完全不同。一个数字本身没有意义，只有它对应的评价目标才有意义。

1.1 你平时看到的评估长什么样

现实中的评估通常包含这些元素：

- **能力基准**：例如 MMLU、MATH、GPQA、Humanity’s Last Exam。
- **偏好排序**：例如 Chatbot Arena 的 pairwise 比较与 ELO。
- **成本视角**：同样的准确率下，推理价格和训练成本可能差很多。
- **使用者视角**：模型不一定要“更聪明”，但一定要“更能卖得出去”或“更适合这个场景”。
- **安全视角**：拒答、越狱、幻觉、双重用途都属于评估范围。

评估并不只看准确率

如果一个模型得分更高，但推理成本高 10 倍、部署风险更大、或者只是在训练数据上记住了测试集，那么它未必是更好的选择。评估必须把成本、鲁棒性和真实使用场景一起考虑进去。

2 如何理解评估

这节课给出的基本框架可以压缩成四个问题：

1. 输入是什么？
2. 如何调用模型？
3. 如何评估输出？
4. 如何解释结果？

2.1 目标先于指标

同一个模型分数，可能对应完全不同的评价目标。

- **用户或公司做购买决策**：该选 Claude、Gemini、Grok，还是别的模型？
- **研究者衡量能力**：模型是否在推进“原始智能”或语言能力？
- **政策和业务分析**：模型带来的收益和危害分别是什么？
- **模型开发者调参**：某个改动是否真的让模型更好？

为什么“我只是评估一下模型”这句话不够

如果不先说清楚目标，评估结果通常会把不同东西混在一起。一个 benchmark 分数可能同时受到 prompt、解码策略、工具使用、训练数据污染、参考答案质量和后处理流程的影响。

2.2 输入、调用、输出、解释

输入部分要问的是：这个评估覆盖了哪些使用场景？有没有尾部难例？输入是不是被改造成更适合模型的形式，比如多轮对话？

调用部分要问的是：是 zero-shot、few-shot，还是 chain-of-thought？有没有工具、RAG 或者其他 agent scaffolding？你评估的是裸模型，还是整个系统？

输出部分要问的是：参考答案是否干净？用什么 metric？像代码任务常用 pass@k，开放式生成可能需要人工偏好、LLM-as-judge 或别的办法。成本要不要算进去？医疗、法律这类场景里，错误是不是有不同代价？

解释部分要问的是：一个 91% 的分数到底意味着什么？能直接上线吗？测试集和训练集有没有重叠？你在评估最终模型，还是某种训练方法？

多轮对话和红队测试会改变输入定义

对于 chat 系统或 red teaming，输入本身往往需要针对模型动态调整。这样更贴近真实行为，但也让不同模型之间的横向对比更难，必须明确规则。

3 Perplexity

在语言模型里，perplexity 是最传统的评估方式之一。把语言模型看成序列分布 $p(x)$ 后，perplexity 可以理解为它给某个数据集 D 赋予的平均困惑程度，常写成

$$\text{PPL}(D) = \left(\frac{1}{p(D)} \right)^{1/|D|}$$

训练时我们最小化训练集上的 perplexity，评估时自然会想看测试集上的 perplexity。

3.1 为什么 perplexity 仍然重要

- 它比 downstream accuracy 更平滑，适合做 scaling law。
- 它是通用的语言建模目标，也是训练目标本身。

- 甚至可以在下游任务上定义条件 perplexity。

perplexity 的局限

如果评估器要信任模型给出的概率分布，那么 perplexity 本身就变成了“要求模型先诚实地报出自己的不确定性”。这比直接看黑盒输出的准确率更脆弱。

3.2 几种“精神上接近 perplexity”的任务

课程里提到，某些 cloze 或补全类任务和 perplexity 的思想很接近，比如 LAMBADA、HellaSwag 等。它们本质上仍然是“给定前文，预测后文”，只是形式更贴近下游任务。

最大化 perplexity 的信念

一种极端观点是：如果模型分布 p 最终逼近真实分布 t ，那就能解决所有任务。但这并不意味着对分布的每一部分都同等值得优化，也不意味着它是最有效的通往强模型的路径。

4 Knowledge Benchmarks

知识类 benchmark 是最常见的一类。它们通常采用多选题，测试的是模型是否掌握某些知识，而不一定是真正意义上的“语言理解”。

4.1 MMLU

MMLU 包含 57 个学科，覆盖数学、历史、法律、道德等内容。它原本是为 few-shot prompting 时代设计的，用来粗略衡量模型在广泛学科上的知识覆盖。

- 优点：覆盖面广，形式标准化。
- 缺点：题目质量参差不齐，且越来越容易被过拟合或污染。
- 本质：更像知识测试，不是纯语言理解测试。

4.2 MMLU-Pro

MMLU-Pro 做了几件事来提升难度：

- 去掉了一些噪声和过于简单的题。
- 把 4 选项扩成 10 选项。
- 用 chain-of-thought 评估，让模型多一点发挥空间。

它的意义在于提醒我们：一个基准一旦变得过于饱和，就不再能很好地区分模型。

4.3 GPQA 和 Humanity's Last Exam

GPQA 强调的是“Google-proof”，问题由博士级别参与者设计。Humanity's Last Exam 则更进一步，试图构造更难、更多模态、更多领域的题目，并通过多轮审核和 frontier model 过滤来降低平庸题目的比例。

基准为何会膨胀

基准题越容易，模型就越快饱和；题越难、越杂、越接近人类研究前沿，越能继续拉开模型差距。但难度提升的同时，构题成本、审核成本和题目质量问题也同步上升。

5 Instruction Following

ChatGPT 之后，很多人关注的不是“知识会不会答”，而是“你会不会照着我说的做”。这类任务更开放，也更接近真实产品交互。

5.1 Chatbot Arena

Chatbot Arena 的核心思想是 pairwise 比较：给用户两个匿名模型回答，让用户选更好的那个，然后基于这些对比算 ELO。它的优点是活的、可持续更新、并且能接纳新模型。

- 优点：接近真实用户偏好。
- 缺点：输入是活流量，比较不稳定，而且可能受到提示词分布影响。
- 结果：更像“谁更受欢迎”，不完全等于“谁更强”。

5.2 IFEval

IFEval 的策略是给 instruction 加上可自动检查的简单约束。这样可以保留一定的开放性，同时让评估可程序化。

IFEval 的思路

把自然语言指令拆成若干个可验证约束，比如长度、格式、包含某些关键词等。它不是在完整意义上评估语义正确性，而是在评估模型是否遵守了显式要求。

5.3 AlpacaEval 和 WildBench

AlpacaEval 主要看 win rate，但一个重要问题是：评判者本身可能偏向某些模型。WildBench 则强调从真实人类对话中抽样，并尝试用更稳的判定方式来减少偏差。

LLM-as-judge 的风险

如果裁判模型和被评模型有共同偏差，或者裁判本身不稳定，那么 leaderboard 可能主要衡量的是“裁判喜欢什么”，而不是“用户真正需要什么”。

6 Agent Benchmarks

一旦任务需要工具调用、长时推理或多轮迭代，传统单轮 benchmark 就不够了。于是出现了 agent benchmark。

6.1 SWE-bench

SWE-bench 要求模型面对真实 codebase 和 issue 描述，最后提交一个 PR，由单元测试来验证是否修复成功。它衡量的是实际软件工程能力，而不只是代码片段生成能力。

6.2 CyBench

CyBench 是安全或攻防相关的 CTF 任务，常用 first-solve time 等方式衡量难度和表现。它提醒我们：有些能力一旦做强了，既能用于正当防御，也能用于攻击。

6.3 MLEBench

MLEBench 更接近数据科学和机器学习工程任务：清洗数据、训练模型、调参、跑实验。它测的不仅是“会不会答题”，而是“会不会完成一个长流程任务”。

Agent benchmark 的特点

Agent benchmark 测的是 **模型 + 脚手架** 的整体效果。这样更接近真实产品，但也更难分清到底是模型本体改进，还是外部 scaffolding 起了作用。

7 Pure Reasoning

课程里讨论的下一个问题是：能不能把 reasoning 从 knowledge 里尽量分离出来？

7.1 ARC-AGI

ARC-AGI 试图测试更接近抽象推理的能力。它的吸引力在于：题目不太依赖背诵事实，而更依赖模式发现、概括和规则迁移。

- ARC-AGI-1 已经很难。
- ARC-AGI-2 更难，说明“纯推理”并不是一个容易孤立出来的维度。

推理和知识并不完全可分

现实模型的表现往往同时依赖知识、语言理解、注意力控制和搜索能力。所谓“纯推理”更多是一种评估理想，而不是一个完全纯净的实验条件。

8 Safety Benchmarks

安全评估不是简单的“会不会拒答”。安全意味着很多东西：有害内容、越狱、双重用途、上下文相关的风险，以及不同地区法律与社会规范的差异。

8.1 HarmBench 与 AIR-Bench

HarmBench 关注违反法律或规范的有害行为；AIR-Bench 则从监管框架和公司政策出发，把风险做了更细的分类。它们都说明：安全不是单一指标，而是一组情境化的约束。

8.2 Jailbreaking

GCG 之类的方法可以自动优化 prompt，诱导模型绕过安全策略。这个现象说明：安全评估不能只依赖模型表面的拒答行为，还要考虑其是否真的抵抗攻击。

8.3 Pre-deployment testing

课程提到美英安全研究机构在模型发布前参与测试。这里的关键不是某个具体流程，而是一个原则：在模型进入公众之前，先做更系统的安全审查。

8.4 什么是安全

安全既包含 **capability**，也包含 **propensity**。

- 一个模型可能有能力做坏事，但平时会拒绝。
- API 模型里，propensity 特别重要。
- 开源模型里，capability 更重要，因为安全层很容易被微调移除。

双重用途

CyBench 之类的任务既可以被看作安全评估，也可以被看作能力评估。一个系统越会做攻防，越需要明确它是在帮助防御，还是在增强攻击能力。

9 Realism

很多标准 benchmark 离真实使用场景非常远。现实用户不是在做考试，而是在“问问题”。

9.1 Quizzing vs Asking

1. **Quizzing**: 用户知道答案，想测试系统。
2. **Asking**: 用户不知道答案，想借助系统得到答案。

后者更接近真实产品使用，也更能体现模型的实际价值。

9.2 Clio 和 MedHELM

Clio 用模型分析真实用户数据，总结人们在问什么；MedHELM 则尝试把医疗评估从标准化考试转向更接近临床实践的任务集合。这里的核心矛盾是：越真实，越有价值，但越容易碰到隐私和合规问题。

真实性和隐私经常冲突

最接近真实流量的数据，往往也是最敏感、最难共享的数据。评估如果只看标准题，会偏离使用场景；如果只看真实数据，又可能受限于隐私和合规。

10 Validity

评估是否有效，是比“分数是多少”更底层的问题。

10.1 Train-test overlap

机器学习最基本的原则是不要在测试集上训练。可是在互联网语料时代，训练数据来自整个网络，是否污染很难彻底判断。

课程提到两条思路：

- **从模型侧推断污染**：利用数据点之间的可交换性等性质去估计重叠。
- **从流程侧要求披露**：让模型提供方报告 train-test overlap 或类似统计。

10.2 Dataset quality

不是所有 benchmark 都是一开始就足够干净。比如 SWE-bench Verified 就是对原始 SWE-bench 做进一步修正得到的更可靠版本。更一般地，很多 benchmark 都需要通过人工或半自动方式不断做“platinum”级别的清洗。

评估有效性与数据质量

如果 benchmark 自身有错误、歧义或污染，那么模型排名的细微差异可能没有你想的那么可信。很多时候，改进 benchmark 和改进模型同样重要。

11 我们到底在评估什么

最后一个核心问题是：你到底在评估 model、system，还是 method？

11.1 方法还是系统

在早期机器学习里，人们更多评估的是 **方法**：给定标准数据集和标准训练流程，谁的算法更好。今天很多场景里，我们评估的是 **模型/系统**：只要最后用户体验更好，内部用了什么 prompt、工具或 scaffold 并不重要。

11.2 为什么这很重要

- 如果你在做研究，可能更关心方法是否普适、是否能推广。
- 如果你在做产品，可能只关心系统最终表现。
- 如果规则不明确，不同团队就会在不同层面“赢得”同一个 benchmark。

11.3 几个例外

课程也提到了一些仍然评估方法的例子：

- **nanoGPT speedrun**：固定数据，比较谁更快达到某个验证损失。
- **DataComp-LM**：给定原始数据，按标准训练管线比较最终效果。

规则的游戏必须说清楚

一个好的评估一定要先声明：输入是什么、允许什么工具、是否允许额外训练、是测最终系统还是测单模型。否则分数本身没有可比性。

12 总结

这节课的结论可以概括成四句话：

- 没有唯一正确的评估方式，评估一定要跟目标对齐。
- 评估不是只看一个总分，要看具体样本和预测。
- 评估至少要同时考虑能力、安全、成本和真实性。
- 一定要讲清楚规则，是在评估方法，还是在评估模型/系统。

本课 takeaways

评估不是“找一个基准然后报分数”，而是一个围绕目标、输入、调用方式、输出和解释来设计的完整过程。真正好的评估，应该能回答你最开始想问的问题，而不是只给你一个看起来漂亮的数字。